

鹰眼长空—使用手册

基于多平台协同的低空无人机智能监测系统

LanGYou

2026-06-28

目录

1 系统概述	2
2 快速启动	2
2.1 环境要求	2
2.2 安装与运行	2
2.2.1 模块单独启动	2
2.2.2 独立模块入口	3
2.3 配置文件	3
3 页面功能	3
3.1 总览 (Dashboard)	3
3.2 预测 (Predict)	4
3.3 分析 (Analysis)	4
3.4 AI 策略 (AI)	4
3.5 数据管理 (Data)	5
4 API 接口	5
4.1 统一响应格式	5
4.2 接口列表	5
4.2.1 刷新轨迹数据	5
4.2.2 加载外部数据	6
4.2.3 清理数据并备份	6
4.2.4 单平台预测	6
4.2.5 全平台预测	6
4.2.6 AI 建议	7
4.2.7 保存报告	7
4.2.8 备份管理	7
4.2.9 分析数据	7
5 数据格式	7
5.1 .dat 轨迹文件	7
5.2 测试数据	8
6 系统架构	8

6.1 双模块设计	8
6.2 分层架构 (reconstruction 模块)	8
6.3 完整目录	8
7 常见问题	9

1 系统概述

鹰眼长空是一个桌面端无人机轨迹监测与预测系统，融合可见光、红外、雷达三种传感平台的轨迹数据，提供 3D 可视化、轨迹预测、AI 策略生成和多维运动学分析功能。

技术栈： Python Flask + pywebview + Three.js + ECharts + 硅基流动 AI

2 快速启动

2.1 环境要求

- Python 3.10+
- 依赖包: flask, pywebview, requests

2.2 安装与运行

安装依赖

```
pip install -r requirements.txt
```

桌面窗口模式

```
python main.py
```

纯 HTTP 服务模式

```
python main.py --headless
```

→ 浏览器访问 <http://127.0.0.1:5000>

2.2.1 模块单独启动

轨迹重建与分析 (默认)

```
python main.py recon # 桌面窗口
```

```
python main.py recon --headless # 纯 HTTP → :5000
```

轨迹识别 (桌面窗口 / 纯 HTTP)

```
python main.py recog # 桌面窗口
```

```
python main.py recog --headless # → :5001
```

同时启动两个模块

```
python main.py all --headless
```

2.2.2 独立模块入口

```
python -m trajectory_reconstruction --headless # → :5000
python -m trajectory_recognition --headless # → :5001
```

2.3 配置文件

编辑 config.json 填入硅基流动 API Key:

```
{
  "siliconflow": {
    "api_key": "your-api-key-here",
    "url": "https://api.siliconflow.cn/v1/chat/completions",
    "model": "Qwen/Qwen2.5-7B-Instruct"
  }
}
```

3 页面功能

系统共有 5 个功能页面，通过顶部导航栏切换。

3.1 总览 (Dashboard)

路由: /

全屏 3D 轨迹视图，是所有检测手段的综合展示页面。

功能:

操作	说明
鼠标左键拖拽	旋转视角
鼠标滚轮	缩放
鼠标右键拖拽	平移
鼠标悬停轨迹点	点球放大 2.5 倍，显示坐标
空格键	播放/暂停动画
□ 播放	从暂停点恢复播放
□ 停止	暂停并保持球体位置
拖动进度条	跳转到指定时间点

覆盖层:

- **左上角—检测平台:** 点击切换各平台显示/隐藏
- **右上角—平台状态:** 实时显示各平台的速度、高度、航向角
- **底部—播放控制条:** 播放/停止、倍速、进度条、预测入口

3D 渲染特性:

- 渐变深色背景 + 线性雾
- 自定义坐标轴（带刻度标记、数字标签、方向箭头）
- CatmullRom 发光轨迹线
- GPU 拖尾粒子系统（2000 粒子/平台）
- ACES 色调映射 + PCF 软阴影

3.2 预测 (Predict)

路由: /predict

基于历史轨迹的线性外推预测。

使用步骤:

1. 选择目标平台（单个或“所有平台”）
2. 调整预测点数和时间步长
3. 点击“开始预测”
4. 预测结果以虚线和半透明球体显示
5. 点击“播放”查看完整轨迹动画

预测算法:

取轨迹最后两点计算速度向量，沿该方向以固定时间步长生成预测点。

$$v = (P_last - P_prev) / \Delta t$$
$$P_pred[i] = P_last + v \times i \times time_step$$

3.3 分析 (Analysis)

路由: /analysis

提供四个多维运动学分析图表:

图表	数据	计算方式
高度变化曲线	Y 坐标序列	直接提取
速度变化曲线	水平速度 (m/s)	水平位移 / 时间差
加速度变化曲线	加速度 (m/s²)	相邻速度差
曲率变化曲线	轨迹曲率 (1/m)	三点法平面曲率

所有图表使用 ECharts 深色主题，支持交互缩放和数据悬浮查看。

3.4 AI 策略 (AI)

路由: /ai

接入硅基流动大模型，生成无人机反制捕捉策略。

使用步骤:

1. 勾选参与分析的检测平台
2. 点击“生成捕捉策略”

3. AI 分析轨迹数据后输出：
 - 运动模式识别
 - 推荐捕捉设备
 - 最佳拦截点坐标
 - 时间窗口建议
4. 可保存报告到 `reports/` 目录

3.5 数据管理 (Data)

路由: `/data`

轨迹数据的上传、备份、恢复与清理。

功能:

操作	说明
上传数据	选择.dat 文件 (每行 x y z t 格式), 创建“自选”平台
查看备份	列出所有备份文件, 支持选择恢复
一键恢复	自动恢复各平台的最新备份
重置数据	重新加载默认测试轨迹
清理数据	清空所有轨迹 (自动备份到 backup/)

数据概览表格: 实时显示各平台的颜色、状态、轨迹点数、时间范围、起止坐标。

4 API 接口

系统提供 11 个 RESTful API 端点, 所有响应均为 JSON 格式。

4.1 统一响应格式

```
{
  "success": true,
  "message": " 描述信息",
  "error": " 错误描述 (仅失败时) "
}
```

4.2 接口列表

4.2.1 刷新轨迹数据

POST `/api/refresh_data`

重新加载 `data/fact/` 下的默认轨迹文件。

4.2.2 加载外部数据

POST /api/load_data

Content-Type: multipart/form-data

参数:

参数	类型	说明
file	File	.dat 格式文件
method_id	String	固定值 "self"

4.2.3 清理数据并备份

POST /api/clear_all_data

清空所有轨迹数据前自动备份到 data/backup/。

4.2.4 单平台预测

POST /api/predict

Content-Type: application/json

请求体:

参数	类型	默认值	说明
method_id	String	-	平台 ID
points	Array	-	历史轨迹点
timestamps	Array	[]	时间戳
num_points	Integer	6	预测点数
time_step	Float	0.5	时间步长 (秒)

响应:

```
{
  "success": true,
  "prediction": [[x,y,z], ... ],
  "pred_times": [t, ... ]
}
```

4.2.5 全平台预测

POST /api/predict_all

对所有可见平台同时进行预测。

4.2.6 AI 建议

POST /api/ai_suggestion

请求体:

```
{
  "methods_data": {
    "visible": { "name": " 可见光", "points": [ ... ], "timestamps": [ ... ] },
    "infrared": { ... }
  }
}
```

4.2.7 保存报告

POST /api/save_report

将 AI 策略保存为报告文件到 reports/ 目录。

4.2.8 备份管理

GET /api/list_backups	列出所有备份
POST /api/restore_backup	恢复指定备份
POST /api/restore_all_backups	一键恢复最新

4.2.9 分析数据

GET /analysis/data

返回各平台的速度、加速度、曲率、高度序列数据。

5 数据格式

5.1 .dat 轨迹文件

每行 4 个空格分隔的数值:

x y z t

字段	含义	单位
x	东西方向位置	米
y	高度	米
z	南北方向位置	米
t	时间戳	秒

5.2 测试数据

系统内置 3 组虚构测试轨迹（各 60 点），存放于 `data/fact/`：

文件	平台	轨迹特征
fact1.dat	可见光	Z 字形起伏飞行
fact2.dat	红外	盘旋上升飞行
fact3.dat	雷达	S 形曲线加速飞行

可替换为真实采集数据，保持相同格式即可。

6 系统架构

6.1 双模块设计

`trajectory_reconstruction/` `trajectory_recognition/`
(轨迹重建与分析) (轨迹识别 — 框架)

□ `data/` □
共享运行时数据

两个模块通过根目录 `data/` 和 `reports/` 共享数据。

6.2 分层架构 (reconstruction 模块)

`views/` (HTTP 薄层) → `services/` (业务编排) → `core/` (领域逻辑)

- **core/**: 纯函数模块 (config / io / prediction / ai / state)
- **services/**: 业务逻辑编排，不依赖 Flask
- **views/**: Flask 蓝图，仅负责 HTTP 解析和 JSON 响应
- **frontend/**: 前端 HTML/CSS/JS

6.3 完整目录

```
drone/
├── main.py                # 统一入口
├── config.json            # 运行配置
├── trajectory_reconstruction/ # 轨迹重建与分析
│   ├── app.py            # Flask 应用工厂
│   ├── __main__.py       # 独立启动入口
│   ├── core/             # 核心领域逻辑
│   │   ├── config/       # 配置管理
│   │   ├── io/           # 数据文件读写
│   │   ├── prediction/   # 轨迹预测算法
│   │   └── ai/           # AI 建议服务
```


└─ state.py	# 共享运行时状态
└─ services/	# 业务逻辑层
└─ views/	# HTTP 视图层
└─ frontend/	# 前端资源
└─ trajectory_recognition/	# 轨迹识别（框架）
└─ app.py	# Flask 应用工厂
└─ __main__.py	# 独立启动入口
└─ features/	# 特征提取
└─ models/	# 识别模型定义
└─ classifier/	# 分类器
└─ data/	# 共享数据
└─ fact/	# 实际轨迹
└─ predict/	# 预测结果
└─ backup/	# 备份
└─ reports/	# 捕捉策略报告
└─ templates/	# 配置模板
└─ docs/	# 文档

7 常见问题

Q: 预测动画球体消失了？

A: 点击 □ 按钮是“暂停”，球体会保持在当前位置。如需彻底清除球体并恢复轨迹显示，刷新页面即可。

Q: 如何更换测试数据？

A: 将真实.dat 文件替换 data/fact/fact1.dat ~ fact3.dat，保持文件名和格式不变，重启程序。

Q: AI 功能不可用？

A: 检查 config.json 中的 siliconflow.api_key 是否为有效的硅基流动 API Key，以及网络连接。

Q: 如何分别启动两个模块？

A: 使用 python main.py recon 启动重建分析，python main.py recog 启动识别。两者可同时运行在不同端口（5000 / 5001）。

Q: 窗口大小如何调整？

A: 桌面窗口可自由拖拽缩放。浏览器模式下直接调整浏览器窗口。3D 视图和图表会自动响应。